

Overview

Linear algebra is core language of ML systems. Features, embeddings, model weights, and activations are vectors/tensors. Efficient learning and inference are mostly matrix operations.

Core Concepts

Scalars, Vectors, Matrices, Tensors

- Scalar: one numeric value.
- Vector: ordered 1-D array.
- Matrix: 2-D table of values.
- Tensor: higher-dimensional generalization.

Matrix Operations

- Addition/subtraction: element-wise, same shape required.
- Matrix multiplication: if $A \in \mathbb{R}^{n \times m}$ and $B \in \mathbb{R}^{m \times p}$, then $AB \in \mathbb{R}^{n \times p}$.
- Transpose: $(A^T)_{ij} = A_{ji}$.

Dot Product, Norm, Cosine Similarity

- Dot product: $a \cdot b = \sum_i a_i b_i$
- Euclidean norm: $\|a\|_2 = \sqrt{\sum_i a_i^2}$
- Cosine similarity:

$$\cos(\theta) = \frac{a \cdot b}{\|a\|_2 \|b\|_2}$$

Eigenvalues, Eigenvectors, SVD (Intuition)

For square matrix A , eigenvector v and eigenvalue λ satisfy:

$$Av = \lambda v$$

SVD factorization:

$$A = U \Sigma V^T$$

SVD helps with compression, denoising, latent-factor discovery, and dimensionality reduction.

Linear Algebra in ML/AI

Linear Regression in Matrix Form

Prediction:

$$\hat{y} = X\theta$$

Normal equation:

$$\theta = (X^T X)^{-1} X^T y$$

Embeddings and Similarity Search

Documents/users/items are vectors in latent space. Retrieval ranks candidates by cosine similarity or dot product.

Neural Network Layers

Dense layer:

$$h = \sigma(Wx + b)$$

where W is weight matrix, x input vector, and σ non-linearity.

Common Pitfalls

- Shape mismatch ((n,m) with incompatible (k,)).
- Row/column confusion causing silent broadcasting bugs.
- Direct matrix inverse on poorly conditioned matrices.
- Feature scaling issues that destabilize numeric operations.

Business Use Cases

- Recommendation systems with user-item embeddings.
- Semantic search and RAG retrieval ranking.
- Dimensionality reduction for analytics and monitoring.

Business Case Studies & Exceptions

Case 1: Embedding Search Quality Drop

New domain docs shifted embedding distribution. Nearest-neighbor quality fell despite high cosine scores.

Mitigation: - Re-embed with domain-adapted model. - Add metadata filters/reranking. - Monitor precision@k by business segment.

Case 2: Linear Solve Instability

Highly collinear features produced unstable inverse in linear regression.

Mitigation: - Standardize features. - Prefer pseudo-inverse or regularization. - Track condition number and alert on instability.

Interview Questions & Answers

1. Q: Dot product vs cosine similarity?

A: Dot product mixes magnitude and direction; cosine normalizes by norms and measures directional alignment.

2. Q: Why matrix multiplication central in ML?

A: Linear transformations in linear models and neural layers are matrix multiplications.

3. Q: Shape rule for matrix multiplication?

A: Inner dimensions must match; output keeps outer dimensions.

4. Q: Why normalize vectors before similarity?

A: Prevent large-magnitude vectors from dominating similarity score.

5. Q: Eigenvector intuition?

A: Direction preserved by transformation, scaled by eigenvalue.

6. Q: Why is SVD useful?

A: Stable decomposition for compression, denoising, and latent structure extraction.

7. **Q: When prefer cosine over Euclidean for embeddings?**
A: When direction matters more than magnitude.
8. **Q: Why can direct inverse be risky?**
A: Near-singular matrices amplify numerical error.
9. **Q: Give production bug example from shapes.**
A: Batch dimension dropped accidentally; broadcast produced wrong predictions silently.
10. **Q: Where linear algebra appears in neural nets?**
A: Every dense projection, attention projection, and batch operation uses matrix math.